CASE STUDY 3 – HOSPITAL BILLING AND PATIENT CLASSIFICATION

1. Problem Statement

Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

2. Objectives

- Calculate a patient's total bill from multiple charge categories.
- Validate that entered charges are non-negative.
- Apply concession rules for senior citizens or low-income patients.
- Classify payment status using the final amount.

3. Assumptions

- Consultation, diagnostics, and treatment charges cannot be negative.
- Senior citizens receive a 10% concession on the subtotal.
- Low-income patients receive an additional 5% concession after the senior concession, if applicable.
- Payment status is 'Paid' if amount paid covers the final bill; otherwise 'Balance Due'.

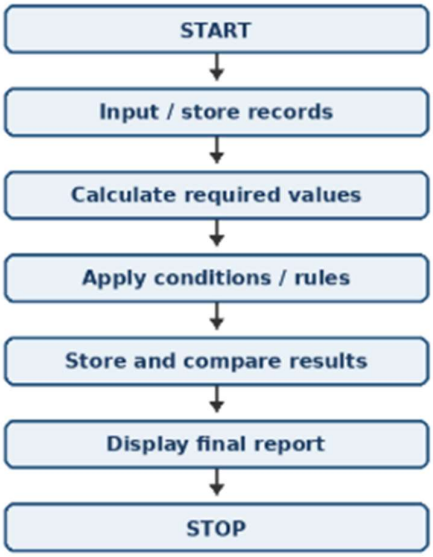
4. Algorithm

- Store patient records as tuples.
- Calculate subtotal for each patient.
- Apply applicable concessions in sequence.
- Calculate final bill and outstanding balance.
- Display bill and payment classification.

5. Pseudocode

- START
- Store patient records.
- FOR each patient: validate all charges and amount paid.
- Calculate subtotal = consultation + diagnostics + treatment.
- Apply senior concession and low-income concession if eligible.
- Calculate final bill and balance.
- Classify as Paid or Balance Due.
- Display patient statement.
- STOP

6. Flowchart



7. Python Source Code

```
patients = [
    ("Ravi", 500, 1200, 3500, "Y", "N", 4500),
    ("Meena", 400, 800, 1800, "N", "Y", 2500),
    ("Kiran", 300, 500, 1000, "N", "N", 1800)
]
for patient in patients:
    name, consultation, diagnostics, treatment, senior, low_income, paid =
patient
    if consultation < 0 or diagnostics < 0 or treatment < 0 or paid < 0:
        print(name, "- Invalid billing data")
    else:
        subtotal = consultation + diagnostics + treatment
        concession = 0
        if senior == "Y":
```

```
    concession = subtotal * 0.10
after_senior = subtotal - concession
if low_income == "Y":
    extra_concession = after_senior * 0.05
else:
    extra_concession = 0
final_bill = after_senior - extra_concession
balance = final_bill - paid
if balance <= 0:
    status = "Paid"
    balance = 0
else:
    status = "Balance Due"
print(
    "Patient:", name,
    "| Subtotal:", round(subtotal, 2),
    "| Concession:", round(concession + extra_concession, 2),
    "| Final bill:", round(final_bill, 2),
    "| Paid:", paid,
    "| Balance:", round(balance, 2),
    "| Status:", status
)
```

8. Test Cases:

Test Case	Patient	Input / Condition	Expected Result
1	Ravi	Subtotal ₹5200; Senior citizen concession 10%; Paid ₹4500	Final Bill: ₹4680; Balance: ₹180; Balance Due
2	Meena	Subtotal ₹3000; Low-income concession 5%; Paid ₹2500	Final Bill: ₹2850; Balance: ₹350; Balance Due
3	Kiran	Subtotal ₹1800; No concession; Paid ₹1800	Final Bill: ₹1800; Balance: ₹0; Paid

9. Sample Output Screenshot:

```
Patient: Ravi | Subtotal: 5200 | Concession: 520.0 | Final bill: 4680.0 | Paid: 4500
| Balance: 180.0 | Status: Balance Due
Patient: Meena | Subtotal: 3000 | Concession: 150.0 | Final bill: 2850.0 | Paid:
2500 | Balance: 350.0 | Status: Balance Due
Patient: Kiran | Subtotal: 1800 | Concession: 0 | Final bill: 1800 | Paid: 1800 |
Balance: 0 | Status: Paid
```

10. Conclusion

The solution validates charges, applies policy-based concessions, and determines outstanding balances. It uses tuples, unpacking, conditions, loops, and arithmetic.